Software Requirements Specification (SRS v2.0)

AURA: Artisan Kitchen & Modern Botanica

Client-Server Architecture (React Native, Node.js, Express, MongoDB)

Document Version: 2.0.0 (Assignment 2 Deliverable)
Academic Term: Fall 2026
Course: Mobile Application Development (CS-3112 / SE-3112)
Student Name: Syed Obaid Ali | **Roll Number:** 9321
Instructor: Dr. Sadaf Tanvir

Executive Summary & Problem Resolution

In **Assignment 1 (A1)**, the AURA Restaurant Application operated strictly as a standalone mobile MVP. All business records—menu catalogs, patron profiles, cart states, table bookings, and kitchen orders—were kept in local mock files and client-side storage (`AsyncStorage`).

> Real-World Problems Observed in A1:
>
> - **Multi-Device Data Desynchronization:** When the Manager changed a price or marked a dish sold-out on their phone, diners on other phones still saw old prices and ordered unavailable dishes.
> - **Cross-Device Order Invisibility:** Orders placed on one device never appeared on another device or after app reinstallation.
> - **Table Double-Booking Hazard:** Two patrons could book the same table at the same time because each phone checked only local state.

> How Assignment 2 (A2) Resolves Every Problem:
>
> - **Single Source of Truth:** Central Express server + MongoDB database connects all client devices.

> - **Instant Price & Availability Sync:** Manager updates via `PUT /api/menu/:id` persist to MongoDB, immediately reflected on customer devices upon pull-to-refresh.
> - **10-Second Live Order Polling:** `OrderTrackingScreen` polls `GET /api/orders/my` every 10 seconds, detecting kitchen status changes without restarts.
> - **Collision-Proof Bookings:** `POST /api/reservations` queries MongoDB and rejects conflicting bookings with **HTTP 409 Conflict**.

---

1. Introduction

1.1. Purpose

This Software Requirements Specification establishes the full functional, database, and architectural requirements for the connected AURA Restaurant Mobile Application. It details the client-server interaction between the React Native frontend, Express.js backend, and MongoDB database.

1.2. Scope

In-Scope (Updated for A2):

1. **Node.js & Express REST API:** Modular backend (`config/` , `models/` , `routes/` , `controllers/` , `middleware/`) running on port 5000.
2. **MongoDB Database:** Production NoSQL database (`restaurant_app`) with Mongoose schemas for User, MenuItem, Table, Reservation, and Order.
3. **Authentication & JWT Security:** Password hashing via `bcryptjs` , signed 24-hour JSON Web Tokens, and `protect` / `managerOnly` middleware.
4. **Dynamic Menu Catalog:** Filterable by category and search, with protected manager-only CRUD endpoints.
5. **Server-Validated Orders:** Server recomputes total prices from database prices and enforces monotonic order progression (`Pending` → `Preparing` → `Ready` → `Served`).
6. **Collision-Proof Reservations:** Verification of table capacity and rejection of duplicate slot bookings with **HTTP 409 Conflict**.
7. **10-Second Polling:** Live status synchronization on customer order tracking screen via `setInterval` .

Out-of-Scope:

- Real credit card payment processing (Stripe, JazzCash, EasyPaisa).
- Native OS push notifications (Apple APNs / Google FCM).
- Real-time GPS courier delivery tracking.

1.3. Definitions, Acronyms, and Abbreviations

Term	Definition
API	Application Programming Interface enabling mobile clients to communicate with the Express server.
REST	Representational State Transfer; standard architecture utilizing HTTP verbs (GET, POST, PUT, PATCH, DELETE) for stateless operations.
Endpoint	A specific URL location (e.g., `/api/menu`) where requests are received and processed.
JSON	JavaScript Object Notation; format used for serializing request payloads and responses.
MongoDB	Document-oriented NoSQL database that stores data in collections of JSON-like BSON documents.
Collection	A grouping of MongoDB documents (analogous to a SQL table, e.g., `menuitems`).
Mongoose	Object Data Modeling (ODM) library providing schema validation, relationships, and queries in Node.js.
Middleware	Express functions that intercept and process requests prior to route controllers (e.g., JWT authentication).
JWT	JSON Web Token; encrypted, signed token used for stateless authentication.
HTTP Status Code	Standardized numerical response code (e.g., 200, 201, 400, 401, 403, 404, 409, 500).

---

5. Database Design (Replaces Section 5 of A1)

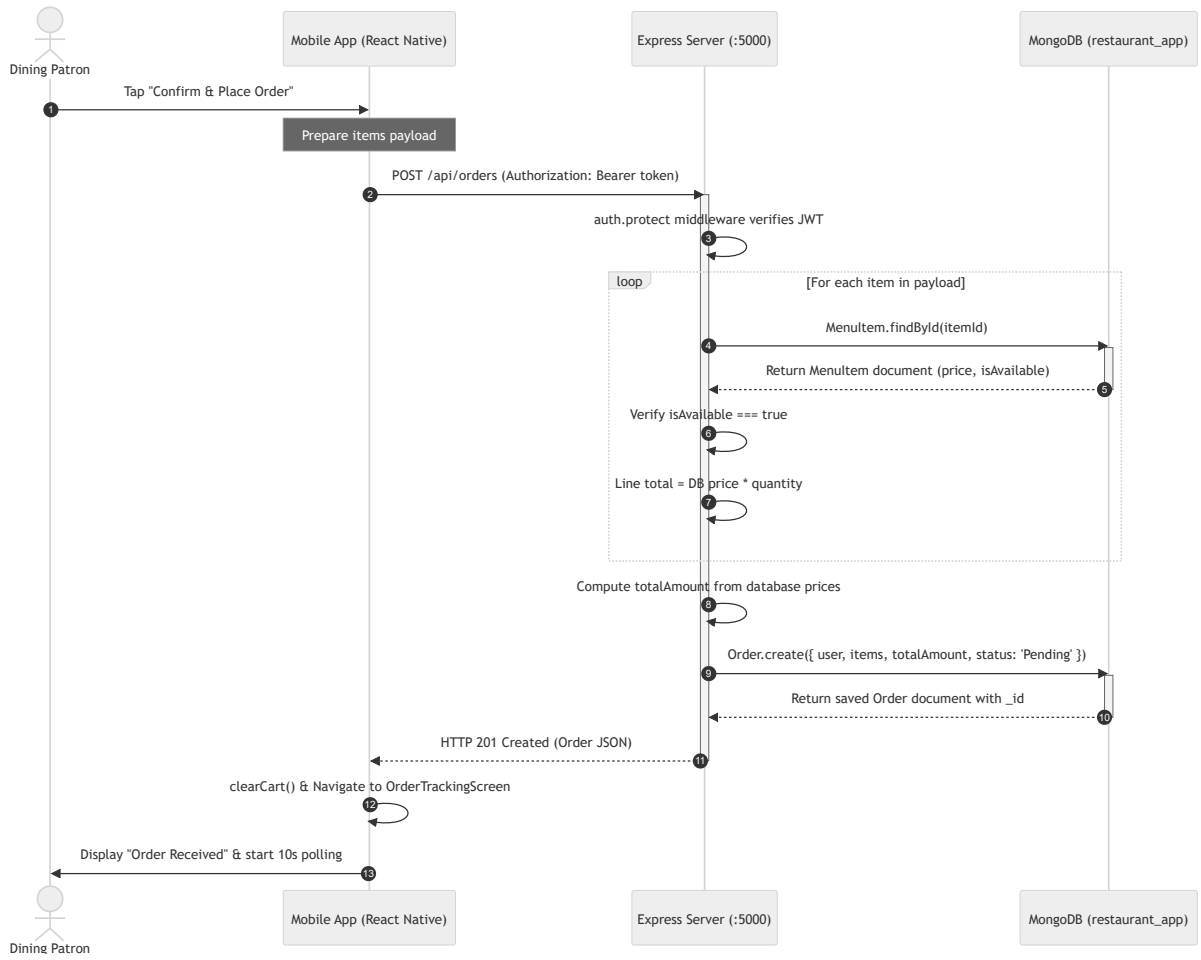
Collection	Fields & Types	Relationships & Business Rules
Users `users`	• `_id` : ObjectId (PK) • `name` : String (Required) • `email` : String (Required, Unique, Lowercase) • `password` : String (Hashed with bcrypt) • `role` : Enum (`customer` , `manager`) • `phone` : String (Optional) • `createdAt` , `updatedAt` : Date	• Unique index on `email` . • Passwords never returned in responses. • Parent of Orders (1:N) and Reservations (1:N).
MenuItems `menuitems`	• `_id` : ObjectId (PK) • `name` : String (Required) • `description` : String (Required) • `price` : Number (Required, Min: 0) • `category` : Enum (Starters, Mains, Desserts, Drinks) • `image` : String (URL) • `isSpecial` : Boolean (Default: false) • `isAvailable` : Boolean (Default: true) • `createdAt` , `updatedAt` : Date	• Price cannot be negative (`min: 0`). • Referenced by `Order.items.menuItem` . • Unavailable items rejected during order placement.
Tables `tables`	• `_id` : ObjectId (PK) • `tableNumber` : String (Required, Unique, e.g. 'A-01') • `name` : String (Required) • `seats` : Number (Required, Min: 1) • `location` : String (Required) • `isWindow` : Boolean (Default: false) • `isAvailable` : Boolean (Default: true)	• Unique constraint on `tableNumber` . • `seats` defines maximum allowable guests. • Referenced by `Reservation.table` .

Reservations `reservations`	• `_id` : ObjectId (PK) • `user` : ObjectId (FK → User) • `customerName` : String (Required) • `phone` : String (Required) • `table` : ObjectId (FK → Table) • `date` : String (YYYY-MM-DD) • `time` : String (HH:mm) • `guests` : Number (Required, Min: 1) • `status` : Enum (Pending, Confirmed, Declined, Cancelled) • `timestamps: true`	• **Collision Prevention (Q7):** Duplicate bookings for same table, date, and time return **HTTP 409 Conflict**. • Table capacity must accommodate guest count.
Orders `orders`	• `_id` : ObjectId (PK) • `user` : ObjectId (FK → User) • `customerName` : String (Required) • `items` : Subdocuments (menuItem, name, price, quantity, specialInstructions) • `totalAmount` : Number (Computed by server) • `orderType` : Enum (Dine-in, Takeaway) • `status` : Enum (Pending, Preparing, Ready, Served, Cancelled) • `timestamps: true`	• **Monotonic State Progression:** Pending → Preparing → Ready → Served. • Cancelled only allowed while Pending. • Total calculated from DB prices (client totals untrusted).

6. Architectural Diagrams

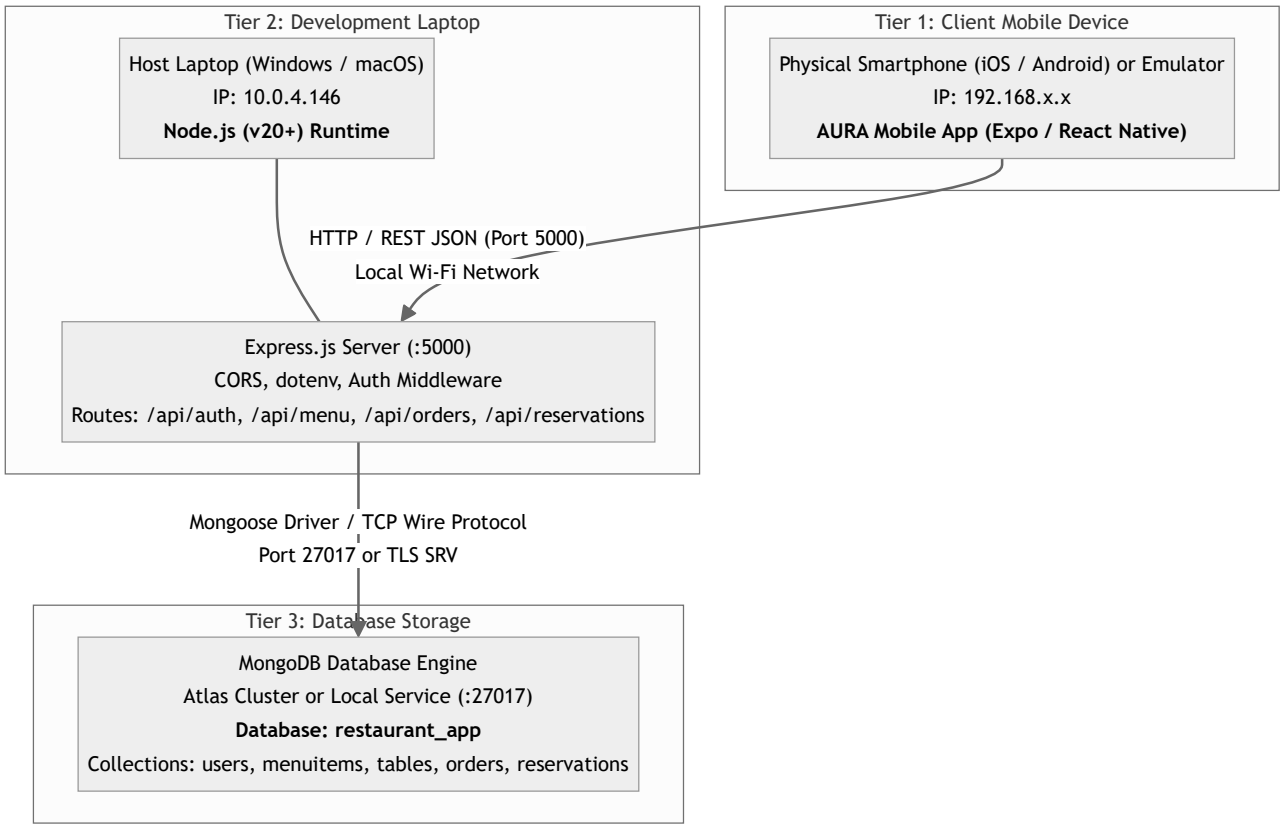
6.1. Sequence Diagram: Customer Places an Order (Q1 Requirement)

Shows the end-to-end order placement flow across the **three required lifelines**: Mobile App, Express Server, and MongoDB.



6.2. Deployment Diagram (Q1 Requirement)

Physical network architecture connecting the mobile device, laptop development server, and MongoDB database.


Tier 2: Development Laptop
Host Laptop (Windows / macOS)
IP: 10.0.4.146
Node.js (v20+) Runtime
Tier 1: Client Mobile Device
Physical Smartphone (iOS / Android) or Emulator
IP: 192.168.x.x
AURA Mobile App (Expo / React Native)
HTTP / REST JSON (Port 5000)
Local Wi-Fi Network
Express.js Server (:5000)
CORS, dotenv, Auth Middleware
Routes: /api/auth, /api/menu, /api/orders, /api/reservations
Mongoose Driver / TCP Wire Protocol
Port 27017 or TLS SRV
Tier 3: Database Storage
MongoDB Database Engine
Atlas Cluster or Local Service (:27017)
Database: restaurant_app
Collections: users, menuitems, tables, orders, reservations

8. API Endpoints Specification (New Section 8)

#	Method	Endpoint Route	Access	Request Body	Response & Status Code
1	`GET`	`/api/health`	Public	None	`200 OK` : `{ status: "ok", time: ... }`
2	`POST`	`/api/auth/register`	Public	`{ name, email, password, role, phone }`	`201 Created` : token & user; `409` if email exists
3	`POST`	`/api/auth/login`	Public	`{ email, password }`	`200 OK` : token & user; `401` on wrong credentials
4	`GET`	`/api/auth/me`	Private	Bearer Token	`200 OK` : current user profile
5	`GET`	`/api/menu`	Public	None (Query: ?category, ?search)	`200 OK` : list of menu items
6	`GET`	`/api/menu/:id`	Public	None	`200 OK` : single item; `404` not found; `400` invalid ID
7	`POST`	`/api/menu`	Manager Only	`{ name, description, price, category, image }`	`201 Created` ; `403 Forbidden` for customers
8	`PUT`	`/api/menu/:id`	Manager Only	`{ price, isAvailable, ... }`	`200 OK` : updated menu item

9	`DELETE`	`/api/menu/:id`	Manager Only	None	`200 OK`: deletion confirmation
10	`POST`	`/api/orders`	Private	`{ items, orderType, tableNumber, notes }`	`201 Created`: order with server-computed total
11	`GET`	`/api/orders/my`	Private	None	`200 OK`: customer's own orders
12	`GET`	`/api/orders`	Manager Only	None	`200 OK`: all restaurant orders
13	`PATCH`	`/api/orders/:id/status`	Manager Only	`{ status: "Preparing"|"Ready"|"Served" }`	`200 OK`; `400` on invalid progression
14	`GET`	`/api/tables`	Public	None	`200 OK`: all dining tables
15	`POST`	`/api/reservations`	Private	`{ tableId, date, time, guests, specialRequests }`	`201 Created`; `409 Conflict` if double-booked
16	`GET`	`/api/reservations/my`	Private	None	`200 OK`: customer's reservations
17	`GET`	`/api/reservations`	Manager Only	None	`200 OK`: all reservations
18	`PATCH`	`/api/reservations/:id`	Manager Only	`{ status: "Confirmed"|"Declined" }`	`200 OK`: updated reservation